Mark's Notes about the UVIS FITS File Format
3 Feb 2026

This is my attempt to accomplish two goals:
- Identify all the geometric quantities that various team members have requested for the new files.
- Identify how this information can be packed into a sensible FITS file, subject to the constraint that it can also be described in a PDS4 label.

This is my proposal for how to proceed, open to any and all questions and comments.

Where example values are shown, I have extracted them from the randomly-chosen sample Titan file:

EUV2013_047_09_33_59_UVIS_181TI_MIDIRTMAP001_CIRS_combined.fits

Below, I do not have anything to say about the HDU entitled DETECTOR_IMG_EUV, because that appears to be a set of derived products that are specific to Titan and may not be relevant to this project.

Note that FITS indexing uses "FORTRAN" index ordering, not "C" fits ordering. The most rapidly-changing axis in a multidimensional array appears first, not last.

hdu[0] = Primary HDU

```
SIMPLE  =                    T / conforms to FITS standard
BITPIX  =                  -32 / array data type is 32-bit float
NAXIS   =                    3 / number of array dimensions
NAXIS1  =                 1024 / number of wavelengths
NAXIS2  =                   64 / number of spatial samples along slit
NAXIS3  =                   33 / number of integration time steps
EXTEND  =                    T
FILENAME= 'EUV2013_047_09_33_59.fits'
PROD_ID = 'EUV2013_047_09_33_59' / PDS product ID
DATE    = '2023-02-04T00:10:43'/ date this file was written (yyyy-mm-dd)
MISSION = 'Cassini'            / mission name = Cassini
INSTRUME= 'UVIS'               / instrument used to acquire data
VERSION =                  1.0 / file version number
OBS_ID  = 'UVIS_181TI_MIDIRTMAP001_CIRS' / Observation ID
MPHASE  = 'XXM'                / Mission Phase (Ground, Prime,Solstice, Equinox)
TARGET  = 'TITAN'              / Target name ("TITAN","SATURN", ...)
ORBNUM  =                  181 / Orbit number = REV number
OBS_ET  =    414279307.7041518 / Observation ephemeris start time secs (J2000)
END_ET  =    414279307.7041518 / Observation ephemeris stop time secs (J2000)
OBS_UTC = '2013-02-16T09:34:00.519' / Observation start UTC date/time
END_UTC = '2013-02-16T11:46:00.519' / Observation end UTC date/time
OBS_SCLK= 'TBD...   '          / Spacecraft clock start count
END_SCLK= 'TBD...   '          / Spacecraft clock stop count
CHANNEL = 'EUV'                / FUV or EUV channel
IMG_XMIN=                    0 / valid window min index on spectral axis
IMG_XMAX=                 1023 / valid window max index on spectral axis
IMG_YMIN=                   15 / valid window min index on slit axis
IMG_YMAX=                   46 / valid window min index on slit axis
IMG_XBIN=                    1 / image binning factor along the spectral axis
IMG_YBIN=                    1 / image binning factor along the slit axis
INT_TIME=                 240. / integration time in seconds
SLITANGL=              xxx.xxx / angle of slit projected on sky, deg
```

```
UNIT    = 'TBD...   '           / calibrated units of data array
COMMENT Calibrated data array
```

Notes

- The quantities currently found in the “CONFIG” HDU should really just be placed in the header, as shown here.
- Don’t include microseconds in times unless they matter. Milliseconds should always be adequate. DATE does not require any fractional seconds.
- OBS_UTC and END_UTC should not contain a time zone. (“_UTC” implies the time zone.)
- I added a place for the spacecraft clock start and stop counts; I think they belong here.
- Slit orientation as projected on the sky has been requested. I assume it has a fixed value for the entire observation, so it can be included in the header as a single value “SLITANGL”.
- We can add other instrument parameters to the header as needed.
- The data object is the calibrated cube as a 3-D array of 4-byte floats. The shape is (NX, NY, NZ). Placing this array (as an IMAGE, not a BINTABLE) in the first HDU makes sense because that is the thing most users will be looking for and will probably expect it here.
- Important strategy question: Most UVIS products are heavily windowed, so most values are zero. We could save a LOT of disk storage space if we only store the values inside the windows. Is it important to save all those extra zeros? I seriously doubt that most users will want them, and the FITS parameters here make the window boundaries clear to any users who do want them.

hdu[“RAW_COUNTS”]

```
XTENSION= ‘IMAGE   ‘           / image extension
BITPIX  =                   16 / array data type is 16-bit int
NAXIS   =                    3 / number of array dimensions
NAXIS1  =                 1024 / number of wavelengths
NAXIS2  =                   64 / number of spatial samples along slit
NAXIS3  =                   33 / number of integration time steps
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
EXTNAME = ‘RAW_COUNTS'         / extension name
COMMENT Raw data array
```

Notes

- The data in this HDU is the raw data array as 16-bit floats (same as in the original PDS3 products). Its indexing is identical to that of the calibrated array in the first HDU. I think this warrants a dedicated HDU.

hdu[“CAL_FACTOR”]

```
XTENSION= ‘IMAGE   '           / image extension
BITPIX  =                  -32 / array data type is 32-bit float
NAXIS   =                    3 / number of array dimensions
NAXIS1  =                 1024 / number of wavelengths
NAXIS2  =                   64 / number of spatial samples along slit
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
EXTNAME = ‘CAL_FACTOR’         / extension name
NULL    =               -1000. / indicates array value is undefined
COMMENT Calibration matrix
```

Notes

- The data in this HDU is the 2-D array of calibration factors as 32-bit floats. This does not include the third (readout step) axis. There’s no reason to embed it inside a one-row, one-column table as is currently the case.
- In the sample file, all values are integers (-1000 or 0,1,2,3), even though it is stored as floats. Why? Is this correct?
- I would guess that -1000 is a “null” value. Correct?

hdu[“SC_GEOM”]

```
XTENSION= 'BINTABLE'           / binary table extension
BITPIX  =                    8 / array data type
NAXIS   =                    2 / number of array dimensions
NAXIS1  =                  TBD / length of dimension 1
NAXIS2  =                  TBD / length of dimension 2
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
TFIELDS =                   23 / number of table fields
EXTNAME = ‘BODY_GEOM'          / extension name
```

TTYPE	TFORM	TDIM	TUNIT	Min	Max	Note
NAME	12A					Name of body inside FOV to which values apply.
SUB_SC_LAT	xxxE	NZ,NT	deg	-90	90	Centric/graphic distinction doesn’t matter for sub- values.
SUB_SC_LON	xxxE	NZ,NT	deg	0	360	
SUB_SOLAR_LAT	xxxE	NZ,NT	deg	-90	90	
SUB_SOLAR_LON	xxxE	NZ,NT	deg	0	360	
SC_ALTITUDE	xxxE	NZ,NT	km	0		
VEL_X_SC_RATE	xxxE	NZ,NT	km/s			
VEL_Y_SC_RATE	xxxE	NZ,NT	km/s			
VEL_Z_SC_RATE	xxxE	NZ,NT	km/s			
VX_SC	xxxE	NZ,NT	km			
VY_SC	xxxE	NZ,NT	km			
VZ_SC	xxxE	NZ,NT	km			
CENTER_RA	xxxE	NZ,NT	deg	0	360	Direction to center of body
CENTER_DEC	xxxE	NZ,NT	deg	-90	90	Direction to center of body
CENTER_PHASE_ANGLE	xxxE	NZ,NT	deg	0	180	Phase angle at center of body
PHI	xxxE	NZ,NT	deg			How is this defined?
RAM_LONGITUDE	xxxE	NZ,NT	deg			
RAM_LATITUDE	xxxE	NZ,NT	deg			Do we need both centric and graphic?
SATURN_LOCAL_TIME	xxxE	NZ,NT	deg			Currently unit is hours. I recommend degrees for consistency with other columns
PROJECTED_DIAMETER	1E		pixel			Diameter of the body in pixels. Bodies smaller than one pixel are omitted from the BODY_GEOM table. This varies with time but I think a single midtime value should suffice.

TTYPE	TFORM	TDIM	TUNIT	Min	Max	Note
CENTER_Y	1E or xE	() or N	pixel			Location of the body center along the slit axis. Expand TDIM if there are multiple center coordinates.
CENTER_Z	1E or xE	() or N	pixel			Location of the body center along the readout axis. Same shape as CENTER_Y.
FULLY_INSIDE_FLAG	1L or xL	() or N		0	1	1 if a body is believed to be fully inside the FOV. Same shape as CENTER_Y.

Notes

- This table should have one row for every body inside the FOV.
- I think we should probably always include Saturn even if it is outside the FOV.
- Note that most of these quantities depend on time but are independent of the pointing direction. Hence, they need dimensions for NZ (number of readouts) and NT (sub-sampling times, typically 3). They do not need a spatial (NY) dimension.
- These quantities are currently found in two HDUs, TARGET_GEOM and SC_GEOM. I have replaced “TARGET” with “CENTER” (referring to the center of the body to which this row applies) in the names, because the new products will describe every body inside the FOV, whether or not it was the target.
- Sub-spacecraft and sub-solar latitude are almost exactly the same in planetocentric and planetographic coordinates. The reason is that -centric latitude is properly calculated where a line to the center of the body pierces the surface, whereas a -graphic latitude is properly calculated where a line of sight to the body is normal to the local surface. As a result, I don’t think we need to tabulate two values for this quantity.
- Do we need to include phase angle at the sub-spacecraft point in addition to that at the body center? The difference is quite small.
- We need to define a lower size cutoff as a fraction of a pixel for which a body is excluded. For example, there’s no clear reason why Aegaeon needs geometric metadata if it is way below the detection threshold.
- CENTER_Y and CENTER_Z are needed so that users can locate unresolved bodies in the data array and perform disk-integrated analysis.
- Due to back-and-forth sweep of the slit, a body can appear multiple times in the data product. There are certainly cases where Saturn appears multiple times. If this occurs, we can expand the TDIM of CENTER_X, CENTER_Y, and FULLY_INSIDE_FLAG to accommodate multiple values.
- Current values are double precision, but that level of precision is not needed by users so it just takes up disk space. I strongly recommend single precision.

hdu[“BODY_GEOM”]

```
XTENSION= 'BINTABLE'           / binary table extension
BITPIX  =                    8 / array data type
NAXIS   =                    2 / number of array dimensions
NAXIS1  =                  TBD / length of dimension 1
NAXIS2  =                  TBD / length of dimension 2
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
TFIELDS =                   14 / number of table fields
EXTNAME = ‘BODY_GEOM'          / extension name
```

TTYPE	TFORM	TDIM	TUNIT	Min	Max	Note
NAME	12A					Name of body inside FOV to which values apply.
LAT_CENTRIC	xxxE	5,NY,NZ,NT	deg	-90	90	
LAT_GRAPHIC	xxxE	5,NY,NZ,NT	deg	-90	90	
LON	xxxE	5,NY,NZ,NT	deg	0	360	
SOLAR_HOUR_ANGLE	xxxE	5,NY,NZ,NT	deg	0	360	
EMISSION_ANGLE	xxxE	5,NY,NZ,NT	deg	0	90	
INCIDENCE_ANGLE	xxxE	5,NY,NZ,NT	deg	0	180	>90 is dark face
PHASE_ANGLE	xxxE	5,NY,NZ,NT	deg	0	180	
DISTANCE	xxxE	5,NY,NZ,NT	km	0		
RAYHEIGHT	xxxE	5,NY,NZ,NT	km	0		
RESOLUTION_FINE	xxxE	5,NY,NZ,NT	km/pixel			Two values needed because of surface orientation at this location
RESOLUTION_COARSE	xxxE	5,NY,NZ,NT	km/pixel			See above. Are these needed?
RING_SHADOW_FLAG	xxxL	5,NY,NZ,NT		0	1	1 if this location on the body is shadowed by the rings.
BEHIND_RING_FLAG	xxxL	5,NY,NZ,NT		0	1	1 if this location on the body is behind the rings.

Notes

- These are the spatial backplanes for Saturn and the satellites.
- This table should have one row for every *resolved* body inside the FOV. If a body is smaller than a pixel, there’s no good reason to generate these quantities; the values in the SC_GEOM table should suffice.
- All values are sampled the same, including five values for the center and corners of each pixel, NY, NZ, and NT (typically 3, for the beginning, middle, and end of each integration period).
- Current backplanes are double precision, but I think single precision ought to suffice.
- Each column needs a specified “TNULL” value to indicate where the backplane is undefined. Anywhere the sample misses the specified body, the array will hold this value.

hdu[“GENERAL_GEOM”]

```
XTENSION= 'BINTABLE'           / binary table extension
BITPIX  =                    8 / array data type
NAXIS   =                    2 / number of array dimensions
NAXIS1  =                  tbd / length of dimension 1
NAXIS2  =                    1 / length of dimension 2
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
TFIELDS =                    7 / number of table fields
EXTNAME = ‘BODY_GEOM'          / extension name
```

TTYPE	TFORM	TDIM	TUNIT	Min	Max	Note
RA	xxxE	5,NY,NZ,NT	deg	-90	90	
DEC	xxxE	5,NY,NZ,NT	deg	-90	90	
TIME_ET	xxxD	NZ,NT	s			SPICE ET in double precision

Notes

- These backplanes do not depend on the body, so this table always has just one row.
- RA and DEC are sampled the same as all other backplanes.
- The TIME_ET backplane only depends on the readout and NT. This one requires double precision.
- Currently, TIME is a separate HDU but I am proposing to move it here.
- Currently, the TIME table also contains an array of strings representing the UTC time. Is this needed? I can’t see a use case. Note that ET and UTC will remain in perfect sync for the duration of any single observation, so the user can always calculate the UTC as (ET - OBS_ET + OBS_UTC).

hdu[“RING_GEOM”]

```
XTENSION= 'BINTABLE'           / binary table extension
BITPIX  =                    8 / array data type
NAXIS   =                    2 / number of array dimensions
NAXIS1  =                  TBD / length of dimension 1
NAXIS2  =                  TBD / length of dimension 2
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
TFIELDS =                   10 / number of table fields
EXTNAME = ‘BODY_GEOM'          / extension name
```

TTYPE	TFORM	TDIM	TU NIT	Min	Max	Note
RING_RADIUS	xxxE	5,NY,NZ,NT	km	0		
RING_LONGITUDE	xxxE	5,NY,NZ,NT	deg	0	360	
RING_AZIMUTH	xxxE	5,NY,NZ,NT	deg	0	360	There’s a request for two values that differ in definition by 90 degrees. Are both values needed?
RING_HOUR_ANGLE	xxxE	5,NY,NZ,NT	deg	0	360	
RING_INTERCEPT_DISTANCE	xxxE	5,NY,NZ,NT	km	0		
RING_EMISSION	xxxE	5,NY,NZ,NT	deg	0	90	
RING_INCIDENCE	xxxE	5,NY,NZ,NT	deg	0	180	>90 is the unlit face. There’s also a request for solar elevation angle -90 to 90 degrees. Is this needed?
RING_PHASE_ANGLE	xxxE	5,NY,NZ,NT	deg	0	180	
IN_SATURN_SHADOW	xxxL	5,NY,NZ,NT		0	1	1 if this location is shadowed by Saturn.
IN_FRONT_OF_SATURN	xxxL	5,NY,NZ,NT		0	1	1 if this location is in front of Saturn.
EDGE_ON_RADIUS	xxxE	5,NY,NZ,NT	km	0		Optional, only appears if ring opening angle is below 5 degrees.
EDGE_ON_ELEVATION	xxxE	5,NY,NZ,NT	km			Ditto.

Notes

- This table always has just one row.
- Values are sampled the same as the BODY_GEOM backplanes.
- There will be some cases where the rings are never in the field of view. In that case, I think we should make TDIM empty, rather than save big arrays that are entirely null-valued.
- The edge-on values are only useful for images of the rings that are nearly edge-on. I propose that we only include these backplanes if the ring opening angle is below 5 degrees (emission between 85 and 95 degrees).

hdu[“KERNELS”]

```
XTENSION= ‘TABLE   ‘           / ASCII table extension
BITPIX  =                    8 / array data type
NAXIS   =                    1 / number of array dimensions
NAXIS1  =                   43 / row width in bytes including <CR><LF>
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
EXTNAME = ‘KERNELS'            / extension name
TBCOL1  =                    2 / start byte (after quote)
TFORM1  = ‘FILENAME’           / SPICE kernel file name
TTYPE1  = ‘A32’                / column width
TBCOL2  =                   36 / start byte (after quote)
TFORM2  = ‘KERNEL_TYPE’        / ‘SPK’, ‘PCK’, ‘INST’, etc.
TFORM2  = ‘A4’                 / column width
```

Notes

- This is the layout I recommend for the list of SPICE kernels.
- There’s one row per kernel file, plus a second column for kernel type. I think is much simpler than the current method of sorting kernels by type and having separate columns for each.
- It uses an ASCII table rather than a binary table, simplifying some use cases.
- As described here, the data array will be a sequence of 43-byte rows of this form:

  ```
  “130321R_SCPSE_13038_13063.bsp    ”,”SPK “<CR><LF>
  ```

- This approach allows the table to be described as an ASCII table in the XML label as well.

hdu[“WAVELENGTH”]

```
XTENSION= ‘IMAGE   ‘           / image extension
BITPIX  =                  -32 / array data type is 32-bit float
NAXIS   =                    1 / number of array dimensions
NAXIS1  =                 1024 / number of wavelengths
PCOUNT  =                    0 / number of group parameters
GCOUNT  =                    1 / number of groups
EXTNAME = ‘WAVELENGTH'         / extension name
UNIT    = ‘Angstrom’
COMMENT Center wavelengths of the spectral samples
```

Notes

- This is the current content of the WAVELENGTH HDU, but as a simple array, not a BINTABLE. The size is NX.